

■ MILEAGE TRACKER

Documento de mejoras — Versión 2.0

Qué cambió, por qué, y qué necesitas configurar tú mismo

Julio 2026

Resumen de la actualización

Esta versión (v2.0) responde a los 5 puntos de mejora solicitados a partir del uso real de la v1.0. A continuación se explica cada cambio: el problema que resolvía, cómo se resolvió técnicamente, y qué archivos del proyecto cambiaron.

#	Solicitado	Estado
1	Actualizar sin perder datos guardados	■ Resuelto (migración de base de datos)
2	Selector de plataformas (DoorDash, Uber, etc.)	■ Resuelto (con íconos genéricos, no logos con copyright)
3	Botón de donación PayPal	■ Resuelto (requiere que pongas tu link)
4	No toma las millas con el cel inactivo	■ Resuelto (Foreground Service)
5	App en segundo plano / minimizada sigue rastreando	■ Resuelto (mismo Foreground Service)
—	Mapa visual offline (Google Maps)	■ Documentado como siguiente paso (v2.1)

MEJORA 1

Actualizar sin perder datos

El problema: al instalar una nueva versión de la app sobre la anterior, si la estructura de la base de datos cambia, Room borra toda la base de datos por defecto y el usuario pierde todos sus viajes guardados.

La solución: se declaró una "Migration" explícita que le dice a Room, paso a paso, cómo transformar la tabla vieja en la nueva (en este caso, agregar la columna "platform") sin borrar ninguna fila existente.

```
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "ALTER TABLE trips ADD COLUMN platform TEXT NOT NULL DEFAULT ''"
        )
    }
}

Room.databaseBuilder(...)
    .addMigrations(MIGRATION_1_2)
    .build()
```

Archivos modificados: data/local/AppDatabase.kt, data/local/TripEntity.kt

Importante para el futuro: cada vez que cambies la estructura de TripEntity, debes subir el número de "version" en @Database y agregar una nueva Migration (ej. MIGRATION_2_3). Si no lo haces, la app se cerrará con un error al abrir en dispositivos que ya tenían datos guardados.

MEJORA 2

Selector de plataformas de trabajo

El problema: al finalizar un viaje, se escribía manualmente el motivo — lento y propenso a errores de tipeo (ej. "doordash" vs "DoorDash" vs "Door Dash").

La solución: un selector de chips con: DoorDash, Amazon Flex, Spark Driver, Uber Eats, Uber, Veho, Instacart, Roadie, Grubhub, y "Otra" (con campo de texto para nombre personalizado).

■ ■ Nota legal importante sobre los logos

Los logos oficiales de DoorDash, Uber, Amazon, etc. son marcas registradas con derechos de autor. Reproducirlos sin licencia — incluso en un proyecto personal de portafolio público en GitHub — no está permitido. Por eso cada plataforma se identifica con su NOMBRE en texto + un COLOR distintivo (inspirado en su identidad de marca) + un ÍCONO GENÉRICO de Material Design (auto, entrega, carrito), que es de uso libre. El resultado visual es limpio y reconocible sin infringir derechos de autor.

Si en algún momento quieres usar los logos reales, la única forma correcta es obtener el asset oficial de cada marca desde su "Brand/Press Kit" y revisar sus términos de uso — varias marcas permiten uso limitado con atribución, pero eso hay que verificarlo caso por caso.

```
val DELIVERY_PLATFORMS = listOf(
    DeliveryPlatform("doordash", "DoorDash", Color(0xFFFF3008), Icons.Default.DeliveryDining),
    DeliveryPlatform("uber_eats", "Uber Eats", Color(0xFF06C167), Icons.Default.RestaurantMenu),
    DeliveryPlatform("other", "Otra", Color(0xFF78909C), Icons.Default.MoreHoriz, isCustom = true),
    // ...
)
```

Archivos nuevos: `util/DeliveryPlatforms.kt`, `ui/components/PlatformSelector.kt`

Archivos modificados: `ui/home/HomeScreen.kt`, `ui/home/HomeViewModel.kt`, `ui/components/TripCard.kt`

MEJORA 3

Botón de donación PayPal

La solución: un botón visible en Inicio y en Resumen que abre el navegador directo a tu link de donación de PayPal.

ACCIÓN REQUERIDA DE TU PARTE: el botón actualmente apunta a un link de ejemplo. Antes de compilar la versión final, edita este archivo:

```
// ui/components/DonateButton.kt
private const val PAYPAL_DONATE_URL = "https://paypal.me/TUUSUARIO" // <-- cámbialo
```

Para conseguir tu link: entra a paypal.com/paypalme, crea tu usuario de PayPal.Me (gratis), y usa la URL que te generen (ej. paypal.me/juanperez).

Archivo nuevo: `ui/components/DonateButton.kt`

MEJORA 4

Millas no se contaban con el cel inactivo

MEJORA 5

App en segundo plano sigue rastreando

Estos dos puntos comparten la MISMA causa raíz y la MISMA solución, así que se explican juntos.

El problema real

En la v1.0, el rastreo GPS vivía dentro del ViewModel de la pantalla Home (HomeViewModel.startTracking()). Android congela agresivamente los ViewModels y corrutinas de una app cuando el usuario minimiza la app, apaga la pantalla, o el sistema necesita liberar batería/memoria (esto se llama "Doze mode" y "App Standby"). El resultado: el GPS dejaba de recibir actualizaciones o las recibía muy espaciadas, y las millas quedaban mal contadas.

La solución: Foreground Service

Se movió TODO el rastreo GPS a un **Foreground Service** (TrackingService.kt) — un tipo especial de componente que Android SÍ mantiene vivo en segundo plano, a cambio de mostrar una notificación persistente y visible ("Rastreando... X millas recorridas"). Esta es la misma técnica que usan apps como Uber, Google Maps o Strava para rastrear rutas con la pantalla apagada.

Cómo se conectan las piezas

- HomeViewModel ya NO escucha el GPS directamente: solo le envía un "Intent" a TrackingService para iniciar/detener.
- TrackingService escucha el GPS con FusedLocationProviderClient, sin importar si la app está visible o no.
- TrackingSessionState (un objeto compartido) es el "puente": el Service escribe ahí las millas actuales, y la pantalla Home lo lee para mostrarlas en tiempo real cuando el usuario vuelve a abrir la app.

```
// HomeViewModel.kt – ya no escucha el GPS, solo ordena start/stop
fun startTracking() {
    val intent = Intent(context, TrackingService::class.java).apply {
        action = TrackingService.ACTION_START
    }
    context.startForegroundService(intent)
}

// TrackingService.kt – el GPS real vive aquí, sobrevive en segundo plano
private fun startTracking() {
    startForeground(NOTIFICATION_ID, buildNotification("0.00 millas"))
    trackingJob = serviceScope.launch {
        locationTracker.trackLocation().collect { point ->
            // actualiza TrackingSessionState y la notificación
        }
    }
}
```

Permiso adicional que verá el usuario

A partir de Android 10, mostrar la ubicación en segundo plano requiere un permiso EXTRA (ACCESS_BACKGROUND_LOCATION) que Android obliga a pedir en un diálogo separado del permiso normal, usualmente llevando al usuario a Ajustes para elegir "Permitir todo el tiempo". La app ahora guía ese flujo automáticamente la primera vez que se presiona "Start Work". Si el usuario lo rechaza, la app sigue funcionando igual mientras la pantalla esté encendida.

Archivos nuevos: *location/TrackingService.kt, location/TrackingSessionState.kt*

Archivos modificados: *AndroidManifest.xml (permisos + declaración del servicio), ui/home/HomeViewModel.kt, ui/home/HomeScreen.kt*

Sobre el mapa offline (pendiente para v2.1)

Aclaración importante: el CONTEO de millas ya no depende de internet — usa GPS satelital puro, que funciona incluso en modo avión con el GPS activado. Lo que sí requeriría internet (o tiles pre-descargados) es MOSTRAR un mapa visual con la ruta dibujada.

Dos caminos para agregarlo en el futuro:

Opción	Costo	Complejidad
Google Maps SDK (con tiles offline)	Requiere facturación activada en Google Cloud y una cuenta de Google Cloud Platform (GCP) Key	Media — tiles offline se descargan y cachean localmente
OSMDroid (OpenStreetMap)	Gratis, sin API key	Media — tiles offline se descargan y cachean localmente

Se recomienda OSMDroid para una v2.1 porque no depende de facturación ni de una cuenta de Google Cloud Platform, lo cual es más simple para un proyecto de portafolio personal.

Recuerda: antes de compilar la versión final, reemplaza el link de PayPal en DonateButton.kt, y prueba el flujo completo de permisos de ubicación en un dispositivo real.

Fin del documento — ¡Tu app ya está lista para la v2.0! ■